

Security Best Practices in Development

STATUS: COMPLETE

Secure coding standards, data protection, validation, and hardened configuration

RUBRIC OBJECTIVE

Demonstrate applied secure coding standards, input validation/sanitization, data protection, and secure configuration - with evidence and purpose for each control.

01 SECURE CONFIGURATION

Restricted FastAPI CORS policy

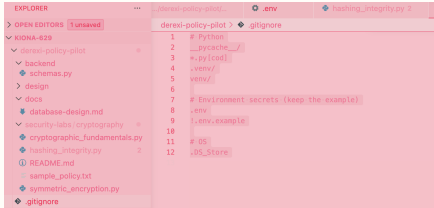
```
59 app.add_middleware(  
60     CORSMiddleware,  
61     allow_origins=[  
62         "http://127.0.0.1:8000",  
63         "http://localhost:8000",  
64     ],  
65     allow_credentials=True,  
66     allow_methods=["GET", "POST", "PATCH"],  
67     allow_headers=["Content-Type", "Authorization"],  
68 )  
69
```

Explicit localhost origins replace wildcard access; methods are limited to GET, POST, and PATCH, and headers to Content-Type and Authorization.

Principle: least privilege / minimize exposed surface.

02 DATA & SECRET PROTECTION

.env excluded from source control



The project keeps .env out of Git while retaining .env.example for safe configuration guidance. Virtual environments and OS noise are also excluded.

Principle: secrets stay outside committed source code.

SECURE CODING STANDARD APPLIED

DeRexi uses explicit allowlists, protected environment files, bounded API inputs, and early rejection of invalid values. The pattern is consistent: accept only what is needed, keep secrets out of source, and stop malformed input before deeper processing.

WHY THESE CONTROLS MATTER

CORS hardening narrows which browser origins, methods, and headers may reach the API. This reduces accidental overexposure during development.

Secret-file exclusion lowers the risk of credentials or connection strings being pushed into Git history, where removal is difficult after exposure.

Together, these controls implement defense-in-depth at the configuration and source-control layers before runtime validation is considered.

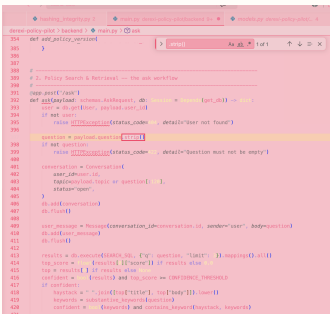
Validation, Normalization & Secure Coding

STATUS: COMPLETE

Evidence from DeRexi's /ask workflow and data-model handling

03 INPUT NORMALIZATION + VALIDATION

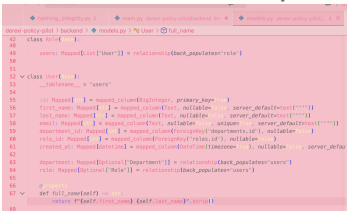
The /ask route cleans user input before downstream use and rejects invalid empty submissions.



payload.question.strip() removes leading/trailing whitespace. If nothing meaningful remains, the API raises HTTP 422 before the value reaches conversation storage or policy search.

04 CONSISTENT DATA HANDLING

Whitespace normalization in model output



The User.full_name property applies .strip() after composing first and last names, keeping displayed values consistent and avoiding stray whitespace.

Scope note: this is normalization, not comprehensive HTML/content sanitization.

05 RUBRIC COVERAGE

Secure coding standards	MET
Input validation	MET
Basic normalization / sanitization	MET
Data protection methods	MET
Secure configuration settings	MET
IMPLEMENTATION OUTCOME	
Rubric #5 requirements are documented with direct evidence.	